

# 100 Regex Ideas for Your Craigslist Cars

---

OPIM 5512 · Module 3 · idea sheet for Lab 3, A06 and your midterm

A **regular expression** (regex) is a tiny search pattern. Instead of searching for the exact text `2013`, you search for **"four digits on a line by themselves"**, and it finds the year in every ad at once. That is how your pipeline turns a messy web page into a row in a table.

**Tonight (Lab 3) you only need ONE of these: the make/model fix (Way 2, idea 10.10).** In **A06** you ship that one line to your pipeline through a pull request. The other 99 are a menu for later: new columns for your **midterm** price model (sections 1, 5, 6 and 8) and text clean-up before Gemini reads your ads in **A07** (section 9).

**How to use this sheet** - 10 sections × 10 ideas. Sections go from easiest to hardest, and so do the ideas inside each one (☆☆☆ easy · ★★☆☆ medium · ★★★★★ stretch). - **Every pattern was tested** on Dr. Wanik's 39 Hartford ads (the Lab 3 backup data). The **Tested** line tells you how many ads it matched and what it grabbed. Your area will be different, and that is the point: test on YOUR cars. - A low hit count is not always a bug. Sometimes the info just is not in most ads (not many sellers write `obo`). - Try them on [regex101.com](https://regex101.com) (pick the **Python** flavor on the left, paste an ad from your `my_cars` folder), or run all 100 on your own cars in the [Try-It notebook](#). - Stuck on a pattern? Paste it into Claude and ask: *"Explain this regex piece by piece, like I'm new to regex."*

## Start here: the regex alphabet

| You type                 | It means                                            | Example                    | Matches                    |
|--------------------------|-----------------------------------------------------|----------------------------|----------------------------|
| <code>abc</code>         | exactly these letters                               | <code>honda</code>         | honda                      |
| <code>.</code>           | any one character (except a line break)             | <code>c.r</code>           | car, cur, c9r              |
| <code>\d</code>          | one digit 0-9                                       | <code>\d\d</code>          | 42                         |
| <code>\w</code>          | one word character (letter, digit, <code>_</code> ) | <code>\w\w\w</code>        | fwd                        |
| <code>\s</code>          | one whitespace (space, tab, line break)             | <code>a\s b</code>         | a b                        |
| <code>\D \W \S</code>    | capital = the OPPOSITE (not digit, etc.)            | <code>\D</code>            | \$                         |
| <code>\n</code>          | a line break                                        | <code>odometer:\n</code>   | the end of the label line  |
| <code>[abc]</code>       | one of these characters                             | <code>gr[ae]y</code>       | gray, grey                 |
| <code>[a-z] [0-9]</code> | one character in a range                            | <code>[A-Z]</code>         | any capital                |
| <code>[^abc]</code>      | one character NOT in the list                       | <code>[^,]</code>          | anything but a comma       |
| <code>+</code>           | one or more of the thing before                     | <code>\d+</code>           | 7, 170000                  |
| <code>*</code>           | zero or more                                        | <code>\d*</code>           | (nothing), 12              |
| <code>?</code>           | optional (zero or one)                              | <code>miles?</code>        | mile, miles                |
| <code>{4} {1,3}</code>   | exactly 4 / between 1 and 3                         | <code>\d{4}</code>         | 2013                       |
| <code>A B</code>         | A or B                                              | <code>obo firm</code>      | obo, firm                  |
| <code>( )</code>         | a capture group: the part you want back             | <code>(\d+) miles</code>   | group(1) = 170000          |
| <code>(?: )</code>       | a group that does NOT capture                       | <code>(?:19 20)\d\d</code> | 2013                       |
| <code>^ \$</code>        | start / end of a line (with <code>re.M</code> )     | <code>^\d{4}\$</code>      | a line that is only a year |
| <code>\A</code>          | start of the whole text                             | <code>\A\d{4}</code>       | the year in the title line |
| <code>\b</code>          | word boundary (edge of a word)                      | <code>\bred\b</code>       | red, not registered        |
| <code>\\$ \. \(\</code>  | backslash = "I mean the real symbol"                | <code>\\$\d+</code>        | \$4000                     |

## The 4 functions you need

| Function                          | What it does                                  | Gives you back                                                                                                                      |
|-----------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>re.search(p, text)</code>   | finds the FIRST match                         | a match object <code>m</code> (or <code>None</code> ): <code>m.group(0)</code> = whole match, <code>m.group(1)</code> = first group |
| <code>re.findall(p, text)</code>  | finds EVERY match                             | a list of strings (or a list of tuples, if the pattern has 2+ groups)                                                               |
| <code>re.sub(p, new, text)</code> | finds and replaces                            | the new text                                                                                                                        |
| <code>re.split(p, text)</code>    | cuts the text wherever <code>p</code> matches | a list of pieces                                                                                                                    |

**Flags** go at the end: `re.search(p, text, re.I)`. Combine them with `|`, like `re.I | re.M`. - `re.I` = IGNORECASE ( `honda` also matches `HONDA` ) - `re.M` = MULTILINE ( `^` and `$` work on every line, not just the start and end of the text ) - `re.S` = DOTALL ( `.` can also match a line break )

**Always write patterns as raw strings:** `r"..."`. The `r` stops Python from treating `\n` or `\d` as its own escape codes.

## Try any idea on your own cars (Colab)

After Lab 3 Part 2b has made your `my_cars` folder, paste this into a new cell:

```
import glob
import re

pattern = r"odometer:\n([\d,]+)" # paste any idea from this sheet here
for path in sorted(glob.glob("/content/my_cars/*.txt")):
    with open(path, encoding="utf-8") as f:
        text = f.read()
        m = re.search(pattern, text)
        if m:
            print(path[-40:], "->", m.group(1))
        else:
            print(path[-40:], "-> no match")
```

## The 10 sections

1. The details block (label: value)
2. Year
3. Dates, times and IDs
4. Price
5. Mileage
6. Paint color
7. Location
8. What the seller says (flags for your model)
9. Clean-up and privacy (`re.sub`)

## 10. Make and model (the boss level)

---

### 1. The details block (label: value)

---

The easiest wins. Craigslist prints a tidy block of `label:` lines, and the value is always on the NEXT line. Learn this pattern and you get half the dataset for free.

---

#### 1.1 ★★★ Condition

```
re.search(r"condition:\n(.+)", text)
```

**How it works:** Find the words `condition:`, then `\n` (the line break), then `(.+)` = grab everything on the next line. The parentheses are a **capture group**: `.group(1)` gives you just what is inside them.

**Tested:** 36 of 39 Hartford ads → `good`, `like new`, `excellent`

---

#### 1.2 ★★★ Title status

```
re.search(r"title status:\n(\w+)", text)
```

**How it works:** Same recipe, but `\w+` = one or more **word characters** (letters, digits, underscore). It stops at the end of the word, so you never pick up stray spaces.

**Tested:** 39 of 39 Hartford ads → `clean`, `rebuilt`, `missing`

---

#### 1.3 ★★★ Transmission

```
re.search(r"transmission:\n(\w+)", text)
```

**How it works:** Same recipe again. Once you see the pattern `label:\n(\w+)`, you can copy it for any field.

**Tested:** 39 of 39 Hartford ads → `automatic`, `manual`

---

#### 1.4 ★★★ Drive (fwd / rwd / 4wd)

```
re.search(r"drive:\n(\w+)", text)
```

**How it works:** `\w` includes digits, so `4wd` comes through whole.

**Watch out:** Case matters by default. `drive:` does not match `DRIVE:` unless you add `re.I`.

**Tested:** 38 of 39 Hartford ads → `fwd`, `rwd`, `4wd`

---

#### 1.5 ★★★ Cylinders as a number

```
re.search(r"cylinders:\n(\d+)", text)
```

**How it works:** `\d+` = one or more **digits**. The value line says `4 cylinders`; `\d+` grabs the `4` and stops at the space. Wrap it in `int(...)` and it is ready for a model.

**Tested:** 39 of 39 Hartford ads → `4`, `8`, `6`

---

## 1.6 ★★★ Body type

```
re.search(r"^type:\n(\w+)", text, re.M)
```

**How it works:** `^` = **start of a line** (needs `re.M`, the MULTILINE flag).

**Watch out:** Without `^`, `type:` would also match the end of `prototype:` or `body type:`. Anchors keep you honest.

**Tested:** 35 of 39 Hartford ads → `SUV`, `truck`, `coupe`

---

## 1.7 ★★★ Paint color (only if listed)

```
re.search(r"paint color:\n(\w+)", text)
```

**How it works:** Same recipe, but this field is **optional**: many sellers skip it. When there is no match, `re.search` returns `None`, so check `if m:` before calling `.group(1)`.

**Watch out:** A low hit count is not a broken regex. It is missing data. Know the difference.

**Tested:** 34 of 39 Hartford ads → `red`, `blue`, `black`

---

## 1.8 ★★★ Only the NON-clean titles

```
re.search(r"title status:\n(?!clean)(\w+)", text)
```

**How it works:** `(?!clean)` is a **negative lookahead**: "only continue if the next word is NOT `clean`." It finds the rebuilt, salvage and missing titles, which are exactly the cars a buyer should worry about.

**Tested:** 2 of 39 Hartford ads → `rebuilt`, `missing`

---

## 1.9 ★★★ Every label and value at once

```
re.findall(r"^([a-z ]+):\n(.*?)$", text, re.M)
```

**How it works:** `[a-z ]+` = a **character class**: any lowercase letter or a space, one or more times. With two groups, `re.findall` returns a list of `(label, value)` pairs, and `dict(...)` turns that into a lookup table in one line.

**Watch out:** It also catches `posted:` (a date). Lowercase-only keeps out `Contact Information:`, which starts with a capital.

**Tested:** 39 of 39 Hartford ads → `[('condition', 'good'), ('cylinders', '4 cylind...)]`

---

## 1.10 ★★★ The details block, cut out whole

```
re.search(r"^(?:19|20)\d{2}\n.*\n((?:[a-z ]+:\n.+)+)", text, re.M)
```

**How it works:** `(?:...)` is a group that does **not** capture (it just groups). `(?:[a-z ]+:\n.+)+` = one label line plus one value line, repeated one or more times. The outer `( )` captures the whole block.

**Watch out:** Idea 1.9 is easier to use; this one shows how groups can repeat.

**Tested:** 35 of 39 Hartford ads → `condition:␣good␣cylinders:␣4 cylinders␣drive:...`,  
`condition:␣good␣cylinders:␣8 cylinders␣drive:...`, `condition:␣like new␣cylinders:␣4 cylinders␣dr...`

---

## 2. Year

---

Every ad has the year in 2 or 3 places. The trick is grabbing the RIGHT 4 digits.

---

### 2.1 ★★★ Any 4 digits

```
re.search(r"\d{4}", text)
```

**How it works:** `\d` = a digit; `{4}` = exactly four of them. `re.search` returns the **first** match, which here is the year at the start of the title.

**Watch out:** Lucky! Try `re.findall` on the same pattern: you get the posted date, the rpm, the post id... `\d{4}` is not the same as a year.

**Tested:** 39 of 39 Hartford ads → `2020`, `1966`, `2004`

---

### 2.2 ★★★ Starts with 19 or 20

```
re.search(r"(19|20)\d{2}", text)
```

**How it works:** `(19|20)` = **alternation**: 19 OR 20. Then two more digits. `5600` rpm no longer counts.

**Watch out:** Use `m.group(0)` (the whole match). `m.group(1)` is only what is inside the parentheses, so `'20'`.

**Tested:** 39 of 39 Hartford ads → `2020`, `1966`, `2004`

---

### 2.3 ★★★ Whole numbers only

```
re.search(r"\b(19|20)\d{2}\b", text)
```

**How it works:** `\b` = a **word boundary** (the edge between a word and a non-word). It stops `2020` from matching inside `7974020201`.

**Tested:** 39 of 39 Hartford ads → `2020`, `1966`, `2004`

---

### 2.4 ★★★ Year at the very start of the file

```
re.search(r"^A(\d{4})", text)
```

**How it works:** `^A` = the **start of the whole text**. Line 1 of every file is the page title, and the title starts with the year.

**Tested:** 39 of 39 Hartford ads → `2020`, `1966`, `2004`

---

### 2.5 ★★★ A line that is ONLY a year

```
re.search(r"^((?:19|20)\d{2})$", text, re.M)
```

**How it works:** With `re.M`, `^` and `$` mean the start and end of **each line**. Craigslist puts the year alone on its own line just above the details block.

**Tested:** 39 of 39 Hartford ads → 2020, 1966, 2004

## 2.6 ★★★ Year from the title line

```
re.search(r"^(\d{4}) .+ for sale by owner", text)
```

**How it works:** `.+` = any characters, one or more. Anchoring on the boilerplate `for sale by owner` means you only look at the title.

**Tested:** 39 of 39 Hartford ads → 2020, 1966, 2004

## 2.7 ★★★ The POSTING year (not the car year!)

```
re.search(r"Posted\n(\d{4})-", text)
```

**How it works:** Context decides meaning. `Posted` + line break + 4 digits + a dash gives the year of the ad itself.

**Watch out:** Use it for car age: 2026 minus the car year.

**Tested:** 39 of 39 Hartford ads → 2026

## 2.8 ★★★ Only plausible car years

```
re.search(r"\b(19[5-9]\d|20[0-2]\d)\b", text)
```

**How it works:** `[5-9]` is a **range** inside a character class. This allows 1950-1999 or 2000-2029, and nothing else.

**Tested:** 39 of 39 Hartford ads → 2020, 1966, 2004

## 2.9 ★★★ Year between line breaks, with lookarounds

```
re.search(r"(?<=\n)(?:19|20)\d{2}(?=\n)", text)
```

**How it works:** `(?<=\n)` = **lookbehind** (a line break must come before). `(?=\n)` = **lookahead** (a line break must come after). Neither is included in the match. It is the same result as 2.5, without the MULTILINE flag.

**Tested:** 39 of 39 Hartford ads → 2020, 1966, 2004

## 2.10 ★★★ Do the years disagree?

```
re.findall(r"\b(?:19|20)\d{2}\b", text)
```

**How it works:** `re.findall` returns every year-looking number in the ad. Put them in a `set()`; if the title year and the description year differ, flag the ad. One Hartford seller lists a 1997 Wrangler whose description says 2008!

**Watch out:** Use `(?:19|20)`, not `(19|20)`. With a capture group, `findall` returns only the group, so you would get `'20'` instead of `'2008'`.

**Tested:** 39 of 39 Hartford ads → ['2020', '2026', '2020']

---

## 3. Dates, times and IDs

---

The easy structured stuff: always the same shape, so it is great practice with exact counts.

---

### 3.1 ★★★ The post id

```
re.search(r"post id: (\d+)", text)
```

**How it works:** Literal text `post id:` followed by digits. This is the ad's unique key; use it to remove duplicates.

**Tested:** 39 of 39 Hartford ads → `7974058269`, `7974064302`, `7974037488`

---

### 3.2 ★★★ A date

```
re.search(r"\d{4}-\d{2}-\d{2}", text)
```

**How it works:** 4 digits, dash, 2 digits, dash, 2 digits. The dash is just a normal character here.

**Tested:** 39 of 39 Hartford ads → `2026-09-27`, `2026-09-28`, `2026-09-25`

---

### 3.3 ★★★ Year, month and day as separate groups

```
re.search(r"(\d{4})-(\d{2})-(\d{2})", text)
```

**How it works:** Three capture groups: `.group(1)`, `.group(2)` and `.group(3)`, or all at once with `.groups()`.

**Tested:** 39 of 39 Hartford ads → `('2026', '09', '27')`, `('2026', '09', '28')`, `('2026', '09', '25')`

---

### 3.4 ★★★ A clock time

```
re.search(r"\b\d{2}:\d{2}\b", text)
```

**How it works:** Two digits, colon, two digits. The `\b` stops it from matching the middle of a longer number.

**Tested:** 39 of 39 Hartford ads → `21:01`, `21:21`, `20:06`

---

### 3.5 ★★★ Date AND time after 'Posted'

```
re.search(r"Posted\n(\d{4}-\d{2}-\d{2} \d{2}:\d{2})", text)
```

**How it works:** Glue the two patterns together with a space. `pd.to_datetime(...)` turns the result into a real timestamp.

**Tested:** 39 of 39 Hartford ads → `2026-09-27 21:01`, `2026-09-27 21:21`, `2026-09-27 20:06`

---

### 3.6 ★★★ Named groups

```
re.search(r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})", text)
```

**How it works:** `(?P<name>...)` gives a group a name. Then `m.group('month')` or `m.groupdict()` gives you a ready-made dict.

**Tested:** 39 of 39 Hartford ads → `('2026', '09', '27')`, `('2026', '09', '28')`, `('2026', '09', '25')`

---

### 3.7 ★★★ Hour of posting (a feature!)

```
re.search(r"Posted\n\d{4}-\d{2}-\d{2} (\d{2}):", text)
```

**How it works:** Capture only the hour. Do evening ads sell for more? Now you can check.

**Tested:** 39 of 39 Hartford ads → `21`, `20`, `22`

---

### 3.8 ★★★ How many photos

```
re.search(r"image 1 of (\d+)", text)
```

**How it works:** The gallery says `image 1 of 24`. More photos might mean a more serious seller, so this is a free feature for your price model.

**Watch out:** Ads with a single photo do not show this line, so no match means 1 photo.

**Tested:** 37 of 39 Hartford ads → `10`, `24`, `6`

---

### 3.9 ★★★ A VIN, anywhere

```
re.search(r"\b[A-HJ-NPR-Z0-9]{17}\b", text)
```

**How it works:** A VIN is exactly 17 characters and never uses I, O or Q. The character class `[A-HJ-NPR-Z0-9]` means A-H, J-N, P, R-Z or a digit, which is exactly the VIN alphabet, and `{17}` sets the length.

**Tested:** 4 of 39 Hartford ads (matches hidden: personal or identifying info)

---

### 3.10 ★★★ Run id and post id from a file path

```
re.search(r"scrapes/(\d{14})/([A-Za-z0-9]+)\.txt", text)
```

**How it works:** `\.` is an **escaped** dot (a real period; a bare `.` means any character). The run id is a 14-digit timestamp (YYYYMMDDHHMMSS), and the post id is the file name. Your scraper pulls the same id out of the ad link with `/view/d/[^/]+/([A-Za-z0-9]+)`, where `[^/]+` = anything except a slash.

**Watch out:** Tested on the `source_txt` column of `listings_master.csv`, not on the ad text.

**Tested:** 29 of 29 Hartford file paths → `('20260928155246', '24WkQVAv4khn6QWdTBQHF1')`, `('20260928155246', '4ysVPsYd2KTDr6rbicgnoV')`, `('20260928155246', '7dEEfyBPBV7RpUamCJ258v')`

---

## 4. Price

The dollar sign is a special character, commas break numbers apart, and sellers write prices every way imaginable.

---

## 4.1 ★★★ A dollar sign and digits

```
re.search(r"\$\d+", text)
```

**How it works:** `$` normally means "end of line", so to find a real dollar sign you **escape** it with a backslash: `\$`.

**Watch out:** On `$11,500` this returns just `$11`, because a comma is not a digit. See the next one.

**Tested:** 39 of 39 Hartford ads → `$11`, `$14`, `$7`

---

## 4.2 ★★★ Let commas in

```
re.search(r"\$[\d,]+", text)
```

**How it works:** `[\d,]` = a digit OR a comma. Now `$11,500` comes through whole.

**Tested:** 39 of 39 Hartford ads → `$11,500`, `$14,000`, `$7,600`

---

## 4.3 ★★★ Negotiable?

```
re.search(r"\b(obo|or best offer|best offer)\b", text, re.I)
```

**How it works:** Alternation plus `re.I` (IGNORECASE) catches `OB0`, `obo` and `or Best Offer`. That gives a yes/no feature.

**Watch out:** Order matters inside `( | )`: the longer option `or best offer` comes first, so it wins.

**Tested:** 5 of 39 Hartford ads → `OB0`, `or Best Offer`, `obo`

---

## 4.4 ★★★ Firm price?

```
re.search(r"\b(firm|non[- ]?negotiable)\b", text, re.I)
```

**How it works:** `[- ]?` = an optional dash or space, so it matches `non-negotiable`, `non negotiable` and `nonnegotiable`.

**Tested:** 8 of 39 Hartford ads → `FIRM`, `firm`

---

## 4.5 ★★★ Real thousands formatting

```
re.search(r"\$\d{1,3}(?:,\d{3})*", text)
```

**How it works:** 1-3 digits, then zero or more groups of `,` plus 3 digits (`*` = zero or more). It won't accept junk like `$1,2,3`.

**Tested:** 39 of 39 Hartford ads → `$11,500`, `$14,000`, `$7,600`

---

## 4.6 ★★★ The price line in the header

```
re.search(r"^\$([\d,]+)$", text, re.M)
```

**How it works:** The official price sits alone on its own line. `^` and `$` (with `re.M`) demand that the whole line is the price.

**Watch out:** Here `$` at the end is the anchor, and `\$` at the start is the dollar sign. Same symbol, two meanings.

**Tested:** 39 of 39 Hartford ads → `11,500` , `14,000` , `7,600`

---

## 4.7 ★★★ The price after the title dash

```
re.search(r"-\\n\\$([\\d,]+)", text)
```

**How it works:** In the header, the title is followed by a line that is just `-` , then the price. Using the neighbor as a landmark is a common scraping trick.

**Tested:** 39 of 39 Hartford ads → `11,500` , `14,000` , `7,600`

---

## 4.8 ★★★ Clean it into a number

```
re.sub(r"D", "", "$11,500")
```

**How it works:** `\D` (capital D) = anything that is **not** a digit. Replace all of them with nothing: `re.sub(r"D", "", "$11,500")` gives `11500` . Then use `int(...)` .

**Watch out:** Careful with `$3.000` (European thousands) and `$11,500.99` . Look at your data before you trust a cleaner.

**Result:** `11500`

---

## 4.9 ★★★ Every dollar amount in the ad

```
re.findall(r"\$\s*:? \s*(\d[\d,.]*)", text)
```

**How it works:** `\s*` = any amount of whitespace (including none), and `:?` = an optional colon. That survives messy text like `$:3.000` . `findall` shows when the description price disagrees with the header price.

**Tested:** 39 of 39 Hartford ads → `['11,500']`

---

## 4.10 ★★★ '14k OBO' style prices

```
re.search(r"\b(\d+(?:\.\d)?)\s?k\s*(?:obo|or best offer|firm)", text, re.I)
```

**How it works:** A number, maybe a decimal ( `(?:\.\d)?` ), optional space, `k` , then a price word. Multiply by 1000.

**Watch out:** Without the price word you would also grab `130k` MILES. Context is everything.

**Tested:** 1 of 39 Hartford ads → `14`

---

# 5. Mileage

The odometer field is clean, but half the useful mileage info lives in the free text: `130k` , `88K miles` , `41,534 ORIGINAL MILES` .

---

## 5.1 ★★★ The odometer field

```
re.search(r"odometer:\n(.+)", text)
```

**How it works:** The details-block recipe from section 1. Start here.

**Tested:** 39 of 39 Hartford ads → 170,000 , 84,000 , 21,000

---

## 5.2 ★★★ Just the number

```
re.search(r"odometer:\n([\d,]+)", text)
```

**How it works:** `[\d,]+` keeps digits and commas and stops at anything else.

**Tested:** 39 of 39 Hartford ads → 170,000 , 84,000 , 21,000

---

## 5.3 ★★★ '... miles' in the text

```
re.search(r"([\d,]+) miles", text, re.I)
```

**How it works:** A number, a space, then `miles`, ignoring case, so `MILES` counts too.

**Tested:** 10 of 39 Hartford ads → 170,000 , 21,000 , 122,331

---

## 5.4 ★★★ The 'k' shorthand

```
re.search(r"\b(\d{2,3})\s?k\b", text, re.I)
```

**How it works:** 2-3 digits, optional space ( `\s?` ), then `k` as a whole word. Matches `130k`, `88K` and `180 k`.

**Watch out:** It also catches `14k 0B0` (a price!) and `Oil change every 10-13 k miles` (not the odometer). See 5.10.

**Tested:** 12 of 39 Hartford ads → 13 , 14 , 88

---

## 5.5 ★★★ 'k miles'

```
re.search(r"\b(\d{2,3})\s?k\s+miles\b", text, re.I)
```

**How it works:** Requiring `miles` after the `k` removes the price confusion, but misses ads that write just `130k`.

**Tested:** 7 of 39 Hartford ads → 13 , 88 , 128

---

## 5.6 ★★★ 'ORIGINAL miles', even misspelled

```
re.search(r"([\d,]+)\s+or?i?ginal\s+miles", text, re.I)
```

**How it works:** `?` = the previous character is **optional**. `or?i?ginal` matches `original` and the seller's `ORIGINAL`. Typos are part of real data.

**Tested:** 4 of 39 Hartford ads → 98,000 , 189,000 , 19,925

---

## 5.7 ★★★ Bragging words

```
re.search(r"\b(?:only|just)\s+([\d,]+(?:\s?k)?)\b", text, re.I)
```

**How it works:** `only 21,000` and `ONLY 88K` are low-mileage brags. `(?:...)` groups `only|just` without capturing it.

**Tested:** 6 of 39 Hartford ads → `122,331`, `98,000`, `88K`

---

## 5.8 ★★★ Either format with alternation

```
re.search(r"\b(\d{1,3}(?:,\d{3})+|\d{2,3}\s?k)\s*miles", text, re.I)
```

**How it works:** `( A | B )`: A = comma-formatted (`170,000`), B = k-format (`88K`). Both must be followed by `miles`.

**Tested:** 15 of 39 Hartford ads → `13 k`, `21,000`, `122,331`

---

## 5.9 ★★★ Skip 'miles on the motor'

```
re.findall(r"([\d,]+)\s+miles(?:!\s+on\s+(?:the\s+)?(?:motor|engine))", text, re.I)
```

**How it works:** `(?! ...)` = negative lookahead. `less than 1000 miles on motor` is about a new engine, not the car, so the lookahead throws it out.

**Tested:** 9 of 39 Hartford ads → `['170,000']`

---

## 5.10 ★★★ k-miles that are NOT prices

```
re.search(r"\b(\d{2,3})\s?k\b(?:!\s*(?:obo|or best|firm))", text, re.I)
```

**How it works:** The fix for 5.4: a `k` number that is not followed by a price word. Then `int(m.group(1)) * 1000` gives miles.

**Tested:** 11 of 39 Hartford ads → `13`, `88`, `128`

---

# 6. Paint color

Only some sellers fill in `paint color:`, so the rest of the color info hides in sentences like `BURGUNDY WITH TAN TWEED INTERIOUR`.

---

## 6.1 ★★★ The naive search

```
re.search(r"red", text, re.I)
```

**How it works:** Plain text is a valid regex. It finds `red` ...

**Watch out:** ...and also finds `red` inside `regist-ERED`, `cove-RED` and `tRED`. That is why we need 6.2.

**Tested:** 16 of 39 Hartford ads → `red`

---

## 6.2 ★★★ Whole word only

```
re.search(r"\bred\b", text, re.I)
```

**How it works:** `\b` on both sides = `red` must be a whole word. The most important fix on this sheet.

**Tested:** 11 of 39 Hartford ads → `red`

---

## 6.3 ★★★ The paint color field

```
re.search(r"paint color:\n(\w+)", text)
```

**How it works:** The details-block recipe. The most reliable, when it is there.

**Tested:** 34 of 39 Hartford ads → `red`, `blue`, `black`

---

## 6.4 ★★★ gray or grey

```
re.search(r"\bgr[ae]y\b", text, re.I)
```

**How it works:** `[ae]` = exactly one character, `a` OR `e`. One pattern, both spellings.

**Tested:** 11 of 39 Hartford ads → `Gray`, `GRAY`, `grey`

---

## 6.5 ★★★ A list of colors

```
re.search(r"\b(red|blue|black|white|silver|gr[ae]y|green|burgundy|tan|gold)\b", text, re.I)
```

**How it works:** Alternation inside a group, with word boundaries on the outside so every option is a whole word.

**Tested:** 33 of 39 Hartford ads → `Gray`, `red`, `blue`

---

## 6.6 ★★★ '\_\_\_ exterior'

```
re.search(r"(\w+) exterior", text, re.I)
```

**How it works:** The word right before `exterior`. It finds `Gray exterior` and even `mica exterior` (from `claret mica`).

**Tested:** 2 of 39 Hartford ads → `Gray`, `mica`

---

## 6.7 ★★★ Shade words

```
re.search(r"\b(dark|light|bright|metallic)\s+(red|blue|gr[ae]y|green|silver)\b", text, re.I)
```

**How it works:** Two groups: the shade and the color. `Bright Red` becomes `('Bright', 'Red')`.

**Tested:** 2 of 39 Hartford ads → `('Bright', 'Red')`, `('DARK', 'GRAY')`

---

## 6.8 ★★★ Exterior/Interior pairs

```
re.search(r"\b(\w+)\s*/\s*(\w+)\s+(?:\w+\s+)?interiou?r\b", text, re.I)
```

**How it works:** BLUE/TAN LEATHER INTERIOUR gives ('BLUE', 'TAN'). \s\*/\s\* allows spaces around the slash, (?:\w+\s+)? allows one optional word (leather, cloth, velour), and interiou?r forgives the extra u.

**Watch out:** On Gray exterior / Black cloth interior it returns ('exterior', 'Black'). Close, but regex has no idea what words mean.

**Tested:** 5 of 39 Hartford ads → ('exterior', 'Black'), ('BLUE', 'TAN'), ('BURGUNDY', 'GRAY')

---

## 6.9 ★★★ 'COLOR WITH ... INTERIOR' lines

```
re.search(r"^[a-z ]+? with .*interiou?r", text, re.I | re.M)
```

**How it works:** +? is a **lazy** (non-greedy) repeat: it grabs as LITTLE as possible, so the group stops at the first with. DARK GRAY WITH CLOTH INTERIOUR gives DARK GRAY.

**Tested:** 3 of 39 Hartford ads → BURGUNDY, SOLID BODY, DARK GRAY

---

## 6.10 ★★★ Field first, text as backup

```
re.search(r"paint color:\n(\w+)\b(red|blue|black|white|silver|gr[ae]y|burgundy)\b", text, re.I)
```

**How it works:** Two alternatives, each with its own group. re.search tries the field first at each position; m.group(1) or m.group(2) gives whichever one matched.

**Watch out:** Compare the hit count with 6.3: that difference is how many colors you rescued from free text.

**Tested:** 35 of 39 Hartford ads → Gray, red, blue

---

# 7. Location

Where is the car? The title line ends with Town, ST - craigslist, and sellers add their own clues.

---

## 7.1 ★★★ Town from the title line

```
re.search(r" - (.*?) , CT - craigslist", text)
```

**How it works:** .+? (lazy) grabs the town and stops at the first , CT.

**Watch out:** It misses towns in MA or NY. See 'Town AND state' below.

**Tested:** 38 of 39 Hartford ads → Plainville, Bolton, Southington

---

## 7.2 ★★★ The Craigslist area

```
re.search(r"CL\n(\w+)\n>", text)
```

**How it works:** The breadcrumb at the top: `CL`, then the area (`hartford`), then `>`. Useful once the whole class combines datasets!

**Tested:** 39 of 39 Hartford ads → `hartford`

### 7.3 ★★★ A list of towns

```
re.search(r"\b(hartford|new britain|bristol|manchester|enfield|vernon)\b", text, re.I)
```

**How it works:** Plain alternation. Easy to read, but only as good as your list.

**Tested:** 39 of 39 Hartford ads → `hartford`, `New Britain`, `Hartford`

### 7.4 ★★★ Town AND state

```
re.search(r" - ([^\n+), ([A-Z]{2}) - craigslist", text)
```

**How it works:** `[^\n+]` = anything except a comma or a line break. `[A-Z]{2}` = exactly two capital letters, which is a state code.

**Tested:** 39 of 39 Hartford ads → `('Plainville', 'CT')`, `('Longmeadow', 'MA')`, `('Bolton', 'CT')`

### 7.5 ★★★ Neighborhood in parentheses

```
re.search(r"^\(((^\n+)\))$", text, re.M)
```

**How it works:** `\(` and `\)` are **escaped** parentheses (real ones, not a group). In between, `[^\n+]` = anything except a closing paren. It finds `(Bolton)` and the seller's typo `(Plaiville)`.

**Tested:** 34 of 39 Hartford ads → `Plaiville`, `Bolton`, `Southington`

### 7.6 ★★★ Compass towns

```
re.search(r"\b(East|West|South|North) ([A-Z][a-z]+)\b", text)
```

**How it works:** `[A-Z][a-z]+` = one capital letter, then lowercase letters, which is a proper name. It finds East Hartford, West Granby and South Windsor.

**Watch out:** Case-sensitive on purpose, so `south of the border` won't count.

**Tested:** 10 of 39 Hartford ads → `('East', 'Hartford')`, `('West', 'Granby')`, `('South', 'Windsor')`

### 7.7 ★★★ 'Located in ...'

```
re.search(r"located in ([A-Z][\w ]+?)(?:[.,\n]|$)", text, re.I)
```

**How it works:** Grab the words after `located in` up to the first period, comma or line break. `(?:[.,\n]|$)` is a stopping point we do not keep.

**Tested:** 3 of 39 Hartford ads → `Avon`, `South Windsor`, `CT`

---

## 7.8 ★★★ 'Location : ...' lines

```
re.search(r"^location\s*:\s*(.+)$", text, re.I | re.M)
```

**How it works:** `\s*` around the colon forgives `Location : Bristol` as well as `Location:Bristol`.

**Tested:** 1 of 39 Hartford ads → `Bristol/ Plainville / New Britain CT`

---

## 7.9 ★★★ Split a list of towns

```
re.split(r"\s*/\s*", "Bristol/ Plainville / New Britain CT")
```

**How it works:** `re.split` cuts a string wherever the pattern matches. Slashes with any spacing become a clean list of towns.

**Result:** `['Bristol', 'Plainville', 'New Britain CT']`

---

## 7.10 ★★★ In-state or out-of-state?

```
re.search(r", (?!CT\b)([A-Z]{2}) - craigslist", text)
```

**How it works:** Negative lookahead again: a state code that is NOT `CT`. It flags the cars you would have to drive to Massachusetts to see.

**Tested:** 1 of 39 Hartford ads → `MA`

---

# 8. What the seller says (flags for your model)

---

Turn free text into yes/no features: new parts, problems, owners, emissions. This is feature engineering with regex.

---

## 8.1 ★★★ New tires?

```
re.search(r"new tires", text, re.I)
```

**How it works:** The simplest possible feature: 1 if found, 0 if not. `bool(re.search(...))`.

**Tested:** 6 of 39 Hartford ads → `New Tires`, `new tires`, `New tires`

---

## 8.2 ★★★ New tires, brakes or battery

```
re.search(r"\bnew (tires|brakes|battery)\b", text, re.I)
```

**How it works:** One group, three options. `.group(1)` tells you which one was mentioned.

**Tested:** 12 of 39 Hartford ads → `BRAKES`, `BATTERY`, `brakes`

---

## 8.3 ★★★ Problem words

```
re.search(r"\b(rust|dents?|crack|damage|leaks?)\b", text, re.I)
```

**How it works:** `dents?` and `leaks?` make the `s` optional.

**Watch out:** It also matches `Rust Free` and `NO LEAKS`, which are good news! See 'Problems that are NOT denied' below.

**Tested:** 9 of 39 Hartford ads → `LEAKS`, `dents`, `Rust`

---

## 8.4 ★★★ One owner

```
re.search(r"\b(1|one|original) owners?\b", text, re.I)
```

**How it works:** Alternation for the three ways people say it.

**Tested:** 4 of 39 Hartford ads → `1`, `original`

---

## 8.5 ★★★ Everything that is 'new'

```
re.findall(r"\bnew (\w+)", text, re.I)
```

**How it works:** `findall` returns every word that follows `new`. Count them, or look at the most common ones with `collections.Counter`.

**Watch out:** `New Britain` is a town, not a new part. Every regex feature needs a quick look at what it caught.

**Tested:** 17 of 39 Hartford ads → `['Britain']`

---

## 8.6 ★★★ What it NEEDS

```
re.findall(r"\bneeds? (?!(nothing))(\w+(?: \w+)?)", text, re.I)
```

**How it works:** `needs?` matches `need` or `needs`. `(?!nothing)` skips the brag `Needs Nothing`. `(?: \w+)?` optionally grabs a second word: `new alternator`, `some TLC`.

**Tested:** 7 of 39 Hartford ads → `['Just Small']`

---

## 8.7 ★★★ Emissions done

```
re.search(r"emissions?\s+(?:done|ready|passed)|passed\s+(?:ct\s+)?emissions", text, re.I)
```

**How it works:** Two word orders joined by `|`: `emissions done` and `Passed CT Emissions`. `(?:ct\s+)?` = optional `CT`.

**Tested:** 11 of 39 Hartford ads → `Emissions passed`, `passed emissions`, `emissions done`

---

## 8.8 ★★★ Service records count

```
re.search(r"(\d+) service records", text, re.I)
```

**How it works:** A number in front of a phrase. `102 Service Records` becomes the feature `102`.

**Tested:** 3 of 39 Hartford ads → `102`, `60`, `29`

## 8.9 ★★★ Problems that are NOT denied

```
re.search(r"(?<!no )(?<!no fluid )\b(rust|dents?|crack|damage|leaks)\b(?! free)", text, re.I)
```

**How it works:** `(?<!no )` = negative **lookbehind**: skip it if `no` comes right before. `(?! free)` skips `rust free`. Negation is the hardest part of text mining, and part of why A07 hands this job to an LLM.

**Tested:** 5 of 39 Hartford ads → `dents`, `Crack`, `rust`

## 8.10 ★★★ SHOUTING lines

```
re.findall(r"^[A-Z0-9 ,.'!&/()-]{15,}$", text, re.M)
```

**How it works:** `{15,}` = 15 or more characters from the class (capitals, digits, punctuation, no lowercase). Count the ALL-CAPS lines: a writing-style feature that often points to the same repeat seller.

**Tested:** 11 of 39 Hartford ads → `'2008 JEEP WRANGLER'`, `'BLUE/TAN LEATHER INTE...`

# 9. Clean-up and privacy (re.sub)

`re.sub(pattern, replacement, text)` finds AND replaces. It is the tool for tidying text before a model or an LLM sees it, and for scrubbing personal information.

## 9.1 ★★★ Remove commas

```
re.sub(r",", "", "170,000")
```

**How it works:** The simplest `re.sub`: find `,`, replace it with nothing. `170,000` becomes `170000`.

**Result:** `170000`

## 9.2 ★★★ Collapse extra spaces

```
re.sub(r"[ \t]+", " ", "ratrod 327 v8")
```

**How it works:** `[ \t]+` = one or more spaces or tabs. Replace each run with a single space.

**Result:** `ratrod 327 v8`

## 9.3 ★★★ Fix a known typo

```
re.sub(r"\bhyunda\b", "hyundai", "2013 Hyunda Sonata", flags=re.I)
```

**How it works:** Whole-word replace. The `\b` matters: without it you would turn `hyundai` into `hyundaii`!

**Result:** `2013 hyundai Sonata`

---

## 9.4 ★★★ Fix a family of typos

```
re.sub(r"\binteriou?r\b", "interior", "TAN LEATHER INTERIOUR", flags=re.I)
```

**How it works:** `interiou?r` matches both spellings, and both become `interior`.

**Result:** TAN LEATHER interior

---

## 9.5 ★★★ Remove emojis and odd symbols

```
re.sub(r"[^\x00-\x7F]+", "", "Emissions Ready👍🚗")
```

**How it works:** `[^\x00-\x7F]` = any character that is NOT plain ASCII. The checkmarks and car emojis vanish.

**Watch out:** It also deletes accented letters (José) and curly quotes. Fine for a regex feature, but don't do this to text you show people.

**Result:** Emissions Ready

---

## 9.6 ★★★ Keep only the description

```
re.search(r"QR Code Link to This Post\n(.*)\npost id:", text, re.S)
```

**How it works:** `re.S` (DOTALL) lets `.` match line breaks too. The lazy `.*` stops at the FIRST `post id:`. What is left is just the seller's words: a much smaller, cleaner input for an LLM.

**Tested:** 39 of 39 Hartford ads → Selling my 2020 Honda HR-V Sport. Reliable, p..., Custom 1966 c10 tons of hand painted on it pi..., 2004 Pontiac Sunfire Bright Red, 140 HP at 56...

---

## 9.7 ★★★ Cut the header junk

```
re.sub(r"\A.*?QR Code Link to This Post\n", "", text, flags=re.S)
```

**How it works:** Same landmark, the other direction: delete everything from the start of the file (`\A`) up to the landmark.

**Tested:** changes 39 of 39 Hartford ads

---

## 9.8 ★★★ Redact phone numbers

```
re.sub(r"(?<!id: )\b(?:\d{3})?[-.\s]?d{3}[-.\s]?d{4}\b", "[PHONE]", text)
```

**How it works:** Optional parentheses around the area code, optional separators (`[-.\s]?`), then 3 + 4 digits. Replace with `[PHONE]` before you share or store text.

**Watch out:** Without `(?<!id: )` it also "redacts" the 10-digit `post id` in every single ad. Test your cleaner, not just your extractor.

**Tested:** changes 0 of 39 Hartford ads

---

## 9.9 ★★★ Redact email addresses

```
re.sub(r"[\w.+-]+@[ \w-]+\.[\w.]+", "[EMAIL]", text)
```

**How it works:** Word characters, dots, plus signs, then `@`, then a domain with at least one dot. A classic, and a responsible habit.

**Tested:** changes 0 of 39 Hartford ads

---

## 9.10 ★★★ Catch sneaky phone numbers

```
re.search(r"\d*(?:zero|one|two|three|four|five|six|seven|eight|nine)\d*[-\s]?d{3}[-\s]?d{2}[-\s]?d{2}",
```



**How it works:** Some sellers write digits as words to dodge filters (like `86zero-...`). This pattern allows a number word mixed into the digits.

**Watch out:** There is always another trick. Regex catches the patterns you thought of, and that is exactly the case for trying an LLM in A07.

**Tested:** 1 of 39 Hartford ads (matches hidden: personal or identifying info)

---

# 10. Make and model (the boss level)

The field the extractor gets wrong. Build up to it here, then finish it in **Lab 3, Way 2**.

---

## 10.1 ★★★ One brand

```
re.search(r"honda", text, re.I)
```

**How it works:** Plain text plus `re.I`. It works, but you would need one pattern per brand.

**Tested:** 12 of 39 Hartford ads → `honda`, `Honda`

---

## 10.2 ★★★ A list of brands

```
re.search(r"\b(honda|toyota|ford|chevy|chevrolet|bmw|jeep|acura|hyundai|nissan)\b", text, re.I)
```

**How it works:** Alternation with word boundaries. Only as good as your list: it never finds a Jaguar.

**Tested:** 33 of 39 Hartford ads → `honda`, `chevy`, `jeep`

---

## 10.3 ★★★ Both spellings of Chevy

```
re.search(r"\bchev(y|rolet)\b", text, re.I)
```

**How it works:** Put the shared start outside the group and the different endings inside: `chev` + (`y` OR `rolet`).

**Tested:** 4 of 39 Hartford ads → `chevy`, `chevrolet`, `Chevrolet`

## 10.4 ★★★ Hyphen or no hyphen

```
re.search(r"\b(cr-?v|hr-?v|rav-?4|f-?150)\b", text, re.I)
```

**How it works:** `-?` = an optional hyphen. `CRV`, `CR-V` and `cr-v` all match.

**Tested:** 5 of 39 Hartford ads → `hr-v`, `f-150`, `cr-v`

## 10.5 ★★★ First word after the title year

```
re.search(r"\A\d{4} (\w+)", text)
```

**How it works:** Start of file, the year, a space, then capture one word. That is usually the make.

**Watch out:** The title is typed by the seller: `FOR FOCUS`, `Hyunda`. Garbage in, garbage out.

**Tested:** 39 of 39 Hartford ads → `honda`, `chevy`, `Pontiac`

## 10.6 ★★★ Make AND model from the title

```
re.search(r"\A\d{4} (\S+) (\S+)", text)
```

**How it works:** `\S` (capital S) = anything that is NOT whitespace. Unlike `\w`, it keeps the hyphen in `hr-v`.

**Tested:** 39 of 39 Hartford ads → `('honda', 'hr-v')`, `('chevy', 'c10')`, `('Pontiac', 'Sunfire')`

## 10.7 ★★★ The whole title before the boilerplate

```
re.search(r"\A(.+?) for sale by owner", text)
```

**How it works:** Everything from the start of the file up to `for sale by owner`. Good for eyeballing and for feeding an LLM.

**Tested:** 39 of 39 Hartford ads → `2020 honda hr-v sport`, `1966 chevy c10 truck`, `2004 Pontiac Sunfire`

## 10.8 ★★★ The model after a known make

```
re.search(r"\b(?:honda|toyota|ford|jeep|acura)\s+([\w-]+)", text, re.I)
```

**How it works:** Match the make without capturing it, then capture the next word. `[\w-]+` = word characters or hyphens.

**Tested:** 24 of 39 Hartford ads → `hr-v`, `wrangler`, `f-150`

## 10.9 ★★★ Why the current extractor fails

```
re.search(r"\b([A-Z][a-z]+)\s+([A-Z][A-Za-z0-9]+)", text)
```

**How it works:** This is the OLD `MAKE_MODEL_RE` in your pipeline: any capitalized word followed by another capitalized word. The first place that happens on the page is often `Contact Information`. Run it and see!

**Watch out:** The lesson: a regex that works on one example can fail on the real page. Always test on many ads.

**Tested:** 39 of 39 Hartford ads → ('Contact', 'Information'), ('Pontiac', 'Sunfire'), ('New', 'Britain')

---

## 10.10 ★★★ Lab 3 Way 2: the details-block version (you finish it)

```
re.search(r"^(?:19|20)\d{2}\n(____)\s+(____)", text, re.M)
```

**How it works:** Combine idea 2.5 (a line that is only a year) with a line break and two capture groups (make, model). The year line is written as `^(?:19|20)\d{2}\n`. The two groups are your job in Lab 3.

**Tested:** Dr. Wanik's answer matches **39 of 39** Hartford ads. Can yours?

---

## Where these ideas go next

---

- **Lab 3, Way 2:** finish idea 10.10 and put it in `BETTER_MAKE_MODEL_RE`.
- **A06:** ship your best make/model regex to your pipeline (one line, on a branch, then a pull request).
- **A07:** Gemini reads the same ads. Where does it beat your regex (negation, typos, sneaky phone numbers), and where does your regex win (speed, cost, exact fields)?
- **Midterm:** sections 5, 6 and 8 are full of new features for your price model: photos, one owner, new tires, service records, color.